

Assignment 6 Report

GUI-Based Multi-Client Chat Application Using TCP

Name: Aman

Roll No: 241001160007

1. Objective

This assignment extends the TCP multi-client chat application developed in Assignment 5 by replacing the terminal-based client with a graphical user interface (GUI) using Python Tkinter. The existing server and socket communication logic were reused with minimal changes while introducing event-driven programming, multithreading, and a user-friendly interface.

2. GUI Design

The client application consists of two windows:

- Login window for entering the server IP, username, and password.
- Main chat window containing a scrollable chat area, online users panel, message input box, Send button, Disconnect button, and connection status label.

The GUI was organized using Tkinter Frames, Labels, Entry widgets, Buttons, Listbox, and ScrolledText to keep the interface simple and responsive.

3. System Architecture

Clients connect to the TCP chat server through sockets over port 5000. The server authenticates the username, maintains the list of connected users, forwards broadcast and private messages, logs chat activity, and manages client connections. Each client runs a background thread that continuously receives messages while the main thread handles GUI events, ensuring the interface remains responsive.

4. Components Reused from Assignment 5

- TCP socket communication
- Multi-client server implementation
- Broadcast messaging
- Private messaging (/msg)

- Online user management
- Chat history
- Connection logging
- Performance logging

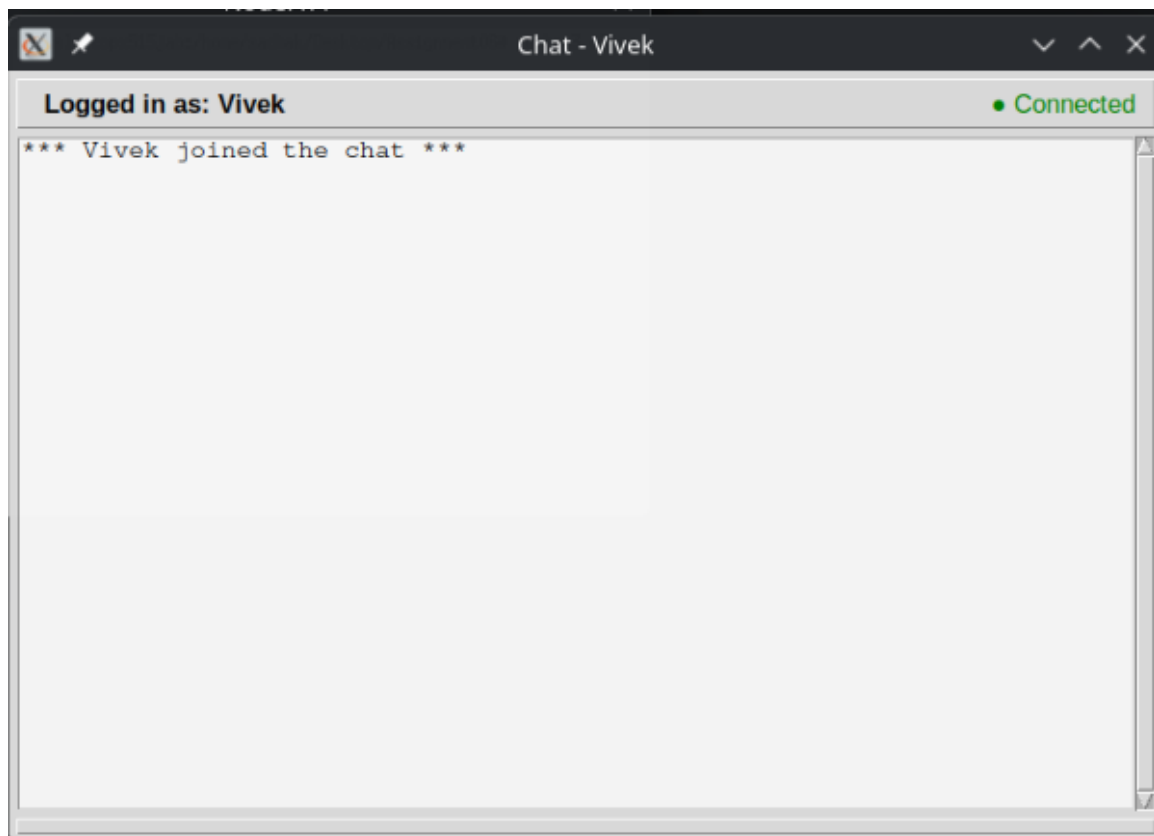
Only the terminal-based client interface was replaced with a Tkinter GUI.

5. GUI Design Decisions

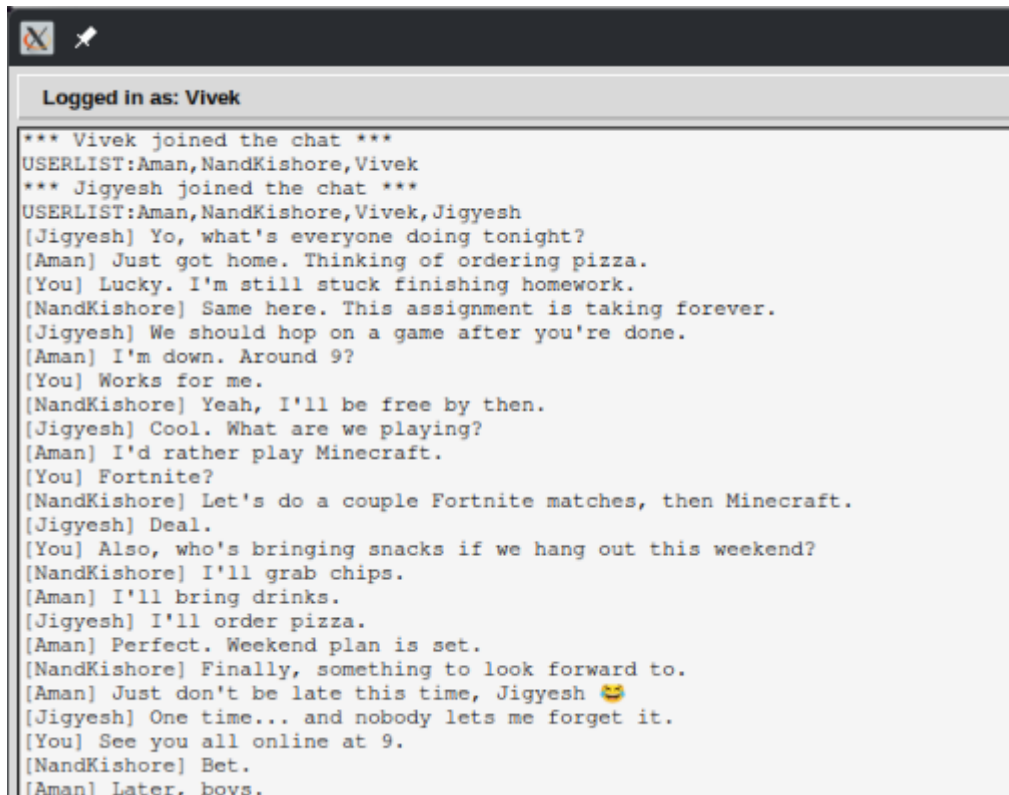
Networking logic was separated from GUI code to improve maintainability and simplify debugging. A background receiver thread was used to prevent the GUI from freezing while waiting for incoming messages. Standard Tkinter widgets were selected because they are lightweight, portable, and part of Python's standard library.

6. Testing Results

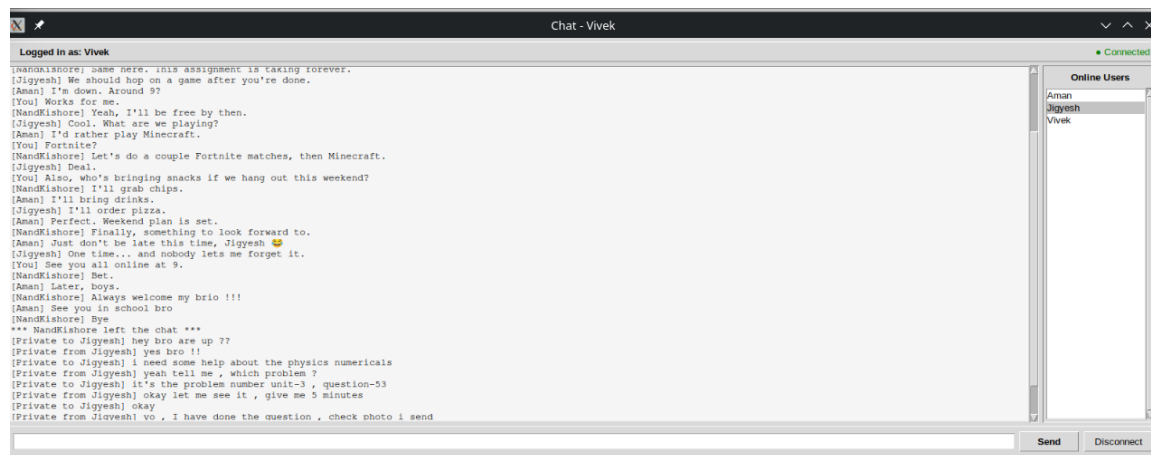
Testing was performed using Mininet with one server and four clients. The following scenarios were verified:



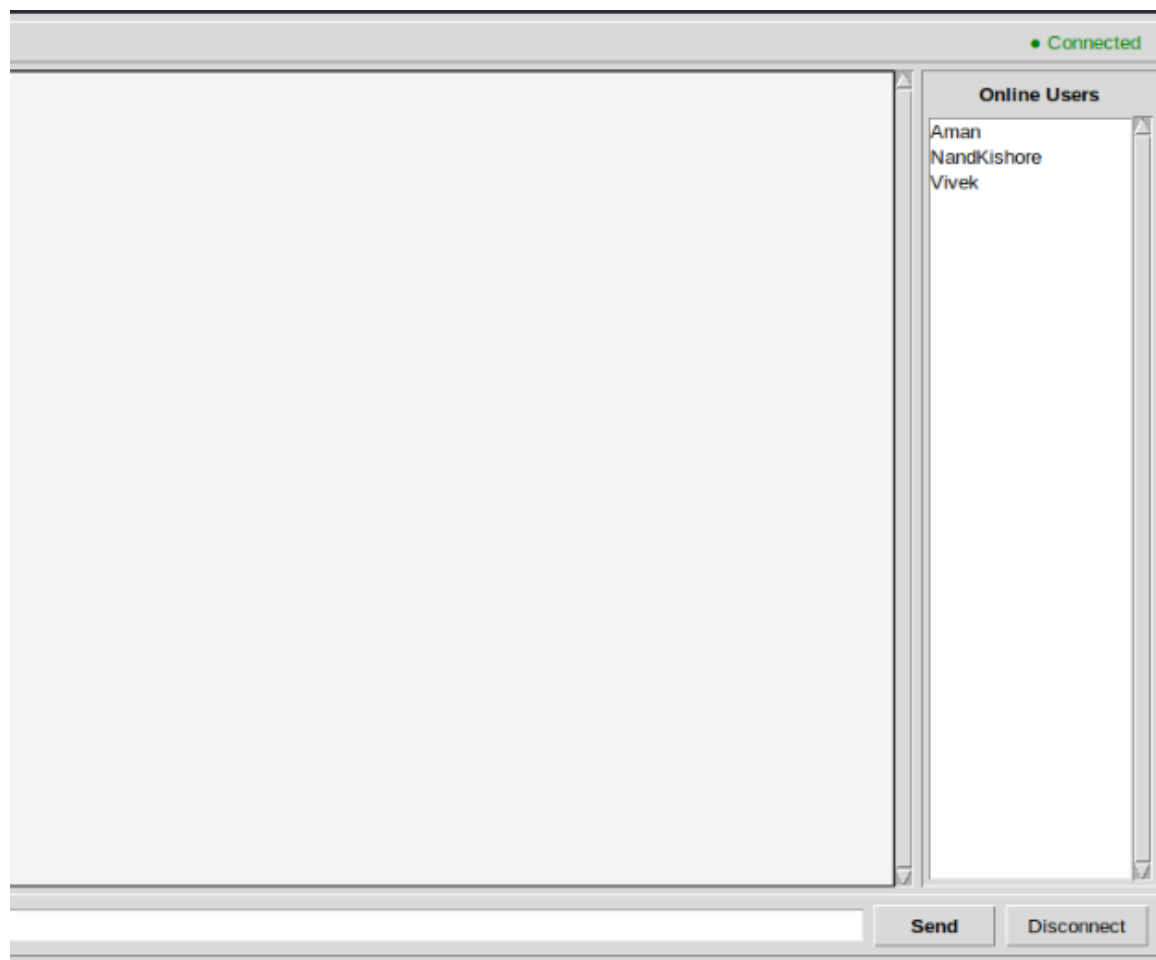
Successful client login



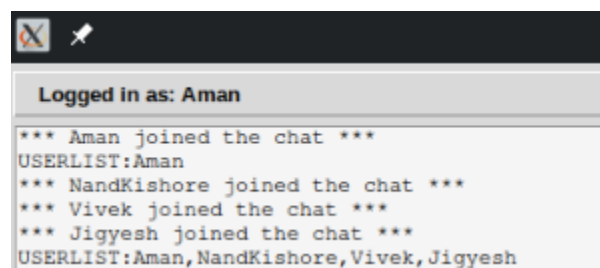
Broadcast messaging



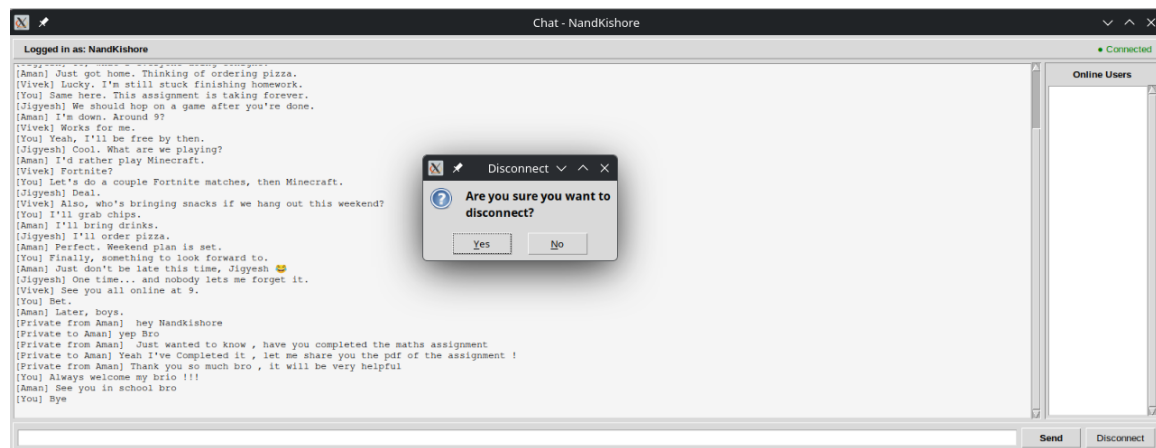
Private messaging



Online user list



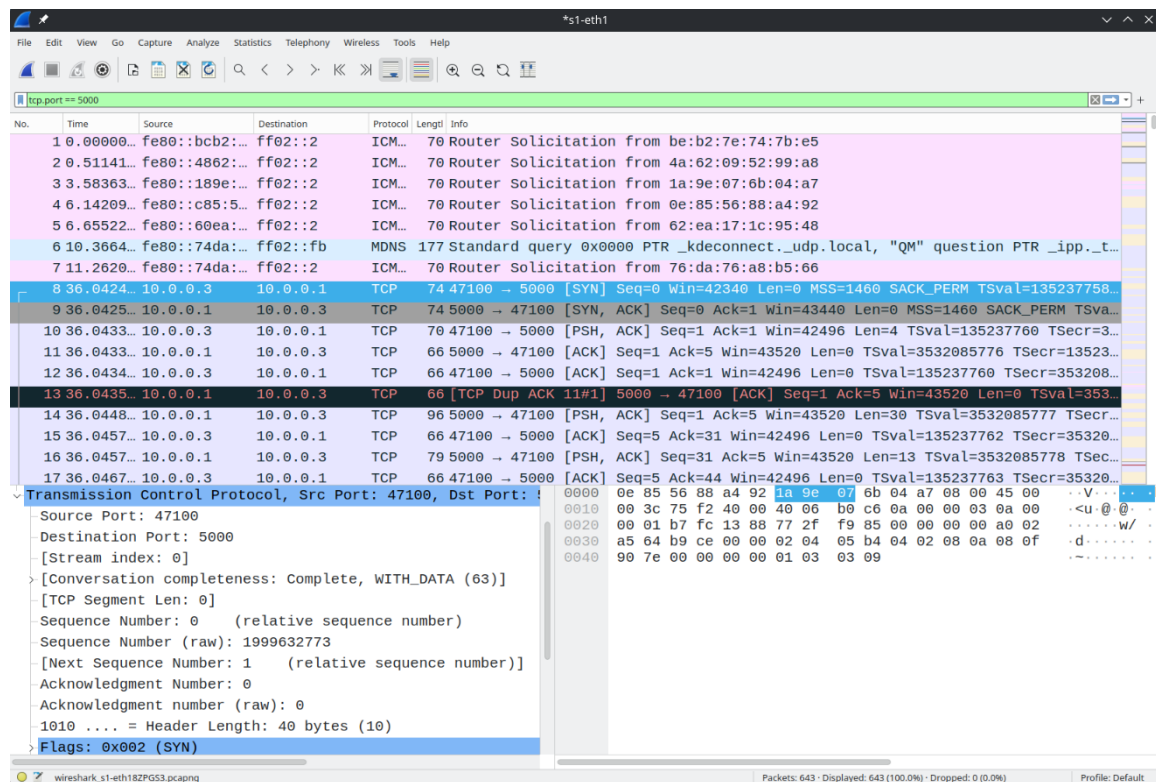
User join notifications



User leave notifications

7. Wireshark Verification

Wireshark was used with the display filter 'tcp.port == 5000'. Captures verified:



TCP client connection

tcp.port == 5000

No.	Time	Source	Destination	Protocol	Length	Info
106	420.863...	fe80::189e::...	ff02::2	ICMPv6	70	Router Solicitation from 1a:9e:07:6b:04:a7
107	424.959...	fe80::bcb2::...	ff02::2	ICMPv6	70	Router Solicitation from be:b2:7e:74:7b:e5
108	433.151...	fe80::c85::...	ff02::2	ICMPv6	70	Router Solicitation from 0e:85:56:88:a4:92
109	441.343...	fe80::60ea::...	ff02::2	ICMPv6	70	Router Solicitation from 62:ea:17:1c:95:48
110	458.367...	fe80::74da::...	ff02::fb	MDNS	177	Standard query 0x0000 PTR _kdeconnect._udp.local, "QM" question PTR _ipp._tcp.local, "QM" qu...
111	505.849...	10.0.0.2	10.0.0.1	TCP	100	58206 → 5000 [PSH, ACK] Seq=8 Ack=73 Win=42496 Len=34 TSval=3524583839 TSecr=393490807
112	505.864...	10.0.0.1	10.0.0.3	TCP	110	5000 → 47100 [PSH, ACK] Seq=240 Ack=5 Win=43520 Len=44 TSval=3532555597 TSecr=135323576
113	505.864...	10.0.0.1	10.0.0.4	TCP	110	5000 → 39622 [PSH, ACK] Seq=197 Ack=12 Win=43520 Len=44 TSval=2133657450 TSecr=3643886522
114	505.864...	10.0.0.1	10.0.0.5	TCP	110	5000 → 39510 [PSH, ACK] Seq=135 Ack=6 Win=43520 Len=44 TSval=207236721 TSecr=2606627272
115	505.865...	10.0.0.3	10.0.0.1	TCP	66	47100 → 5000 [ACK] Seq=5 Ack=284 Win=42496 Len=0 TSval=135707581 TSecr=3532555597
116	505.865...	10.0.0.4	10.0.0.1	TCP	66	39622 → 5000 [ACK] Seq=12 Ack=241 Win=42496 Len=0 TSval=3644270526 TSecr=2133657450
117	505.865...	10.0.0.5	10.0.0.1	TCP	66	39510 → 5000 [ACK] Seq=6 Ack=179 Win=42496 Len=0 TSval=2607011276 TSecr=207236721
118	505.890...	10.0.0.1	10.0.0.2	TCP	66	5000 → 58206 [ACK] Seq=73 Ack=42 Win=43520 Len=0 TSval=393874836 TSecr=3524583839
119	506.878...	fe80::74da::...	ff02::2	ICMPv6	70	Router Solicitation from 76:da:76:a8:b5:66
120	510.974...	4a:62:09:52...	0e:85:56:88...	ARP	42	Who has 10.0.0.1? Tell 10.0.0.5
121	510.974...	0e:85:56:88...	4a:62:09:52...	ARP	42	10.0.0.1 is at 0e:85:56:88:a4:92
122	510.975...	be:b2:7e:74...	0e:85:56:88...	ARP	42	Who has 10.0.0.1? Tell 10.0.0.4

[Calculated window size: 43520]
[Window size scaling factor: 512]
Checksum: 0x1458 [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Time
[Timestamps]
[SEQ/ACK analysis]
TCP payload (44 bytes)
Data (44 bytes)
Data: 5b4a6967796573685d28596f2c207768617427732065760572796f6e52
[Length: 44]

Packets: 643 - Displayed: 643 (100.0%) - Dropped: 0 (0.0%) Profile: Default

Broadcast message transmission

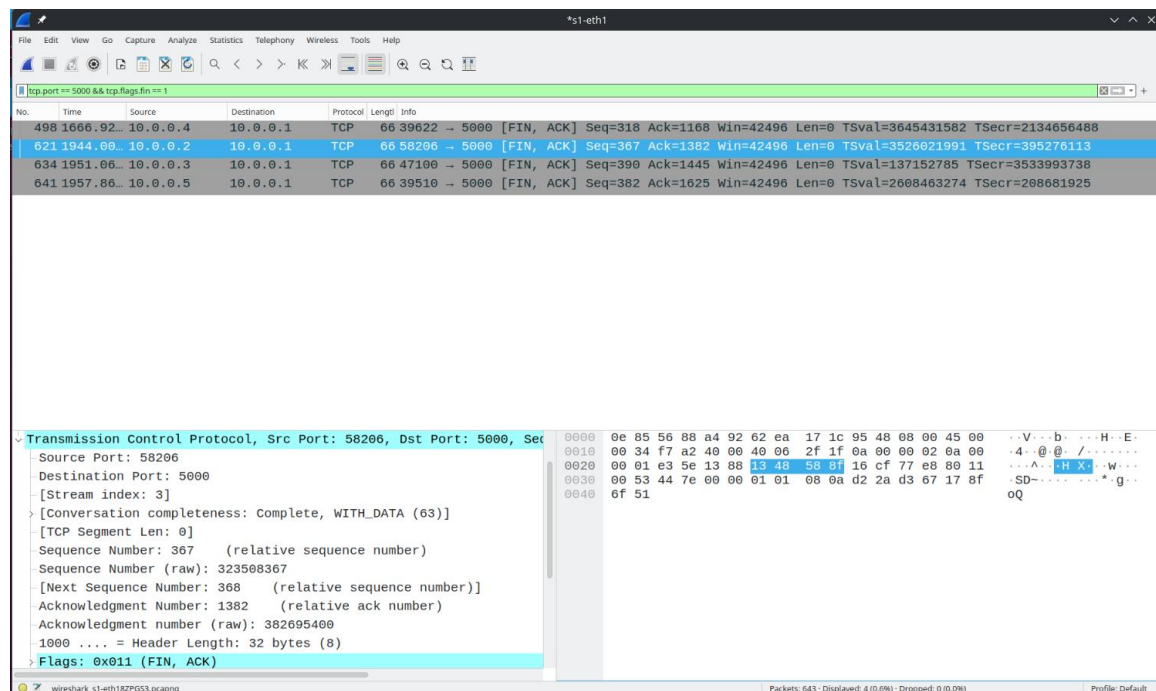
tcp.port == 5000

No.	Time	Source	Destination	Protocol	Length	Info
538	1716.73...	62:ea:17:1c...	0e:85:56:88...	ARP	42	Who has 10.0.0.1? Tell 10.0.0.2
539	1716.73...	0e:85:56:88...	62:ea:17:1c...	ARP	42	10.0.0.1 is at 0e:85:56:88:a4:92
540	1716.73...	4a:62:09:52...	0e:85:56:88...	ARP	42	10.0.0.5 is at 4a:62:09:52:99:a8
541	1716.73...	0e:85:56:88...	62:ea:17:1c...	ARP	42	Who has 10.0.0.2? Tell 10.0.0.1
542	1716.73...	62:ea:17:1c...	0e:85:56:88...	ARP	42	10.0.0.2 is at 62:ea:17:1c:95:48
543	1718.27...	10.0.0.2	10.0.0.1	TCP	87	58206 → 5000 [PSH, ACK] Seq=173 Ack=944 Win=42496 Len=21 TSval=3525796268 TSecr=395080530
544	1718.27...	10.0.0.1	10.0.0.2	TCP	66	5000 → 58206 [ACK] Seq=944 Ack=194 Win=43520 Len=0 TSval=395087222 TSecr=3525796268
545	1718.27...	10.0.0.1	10.0.0.5	TCP	99	5000 → 39510 [PSH, ACK] Seq=1047 Ack=180 Win=43520 Len=33 TSval=208449134 TSecr=2608216996
546	1718.27...	10.0.0.5	10.0.0.1	TCP	66	39510 → 5000 [ACK] Seq=180 Ack=1080 Win=42496 Len=0 TSval=2608223689 TSecr=208449134
547	1718.27...	10.0.0.1	10.0.0.2	TCP	95	5000 → 58206 [PSH, ACK] Seq=944 Ack=194 Win=43520 Len=29 TSval=395087223 TSecr=3525796268
548	1718.27...	10.0.0.2	10.0.0.1	TCP	66	58206 → 5000 [ACK] Seq=194 Ack=973 Win=42496 Len=0 TSval=3525796269 TSecr=395087223
549	1719.29...	fe80::4862::...	ff02::2	ICMPv6	70	Router Solicitation from 4a:62:09:52:99:a8
550	1733.00...	10.0.0.5	10.0.0.1	TCP	124	39510 → 5000 [PSH, ACK] Seq=180 Ack=1080 Win=42496 Len=58 TSval=2608238416 TSecr=208449134
551	1733.00...	10.0.0.1	10.0.0.2	TCP	132	5000 → 58206 [PSH, ACK] Seq=973 Ack=194 Win=43520 Len=66 TSval=395101951 TSecr=3525796269
552	1733.00...	10.0.0.1	10.0.0.5	TCP	132	5000 → 39510 [PSH, ACK] Seq=1080 Ack=238 Win=43520 Len=66 TSval=208463862 TSecr=2608238416
553	1733.00...	10.0.0.5	10.0.0.1	TCP	66	39510 → 5000 [ACK] Seq=238 Ack=1146 Win=42496 Len=0 TSval=2608238417 TSecr=208463862
554	1733.00...	10.0.0.2	10.0.0.1	TCP	66	58206 → 5000 [ACK] Seq=194 Ack=1039 Win=42496 Len=0 TSval=3525910997 TSecr=395101951

Window: 85
[Calculated window size: 43520]
[Window size scaling factor: 512]
Checksum: 0x146b [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Time
[Timestamps]
[SEQ/ACK analysis]
TCP payload (66 bytes)
Data (66 bytes)
Data: 5b507269766174652066726f6d20656976656b5d2069206e65656420736
[Length: 66]

Packets: 643 - Displayed: 643 (100.0%) - Dropped: 0 (0.0%) Profile: Default

Private message transmission



Client disconnection (TCP FIN/ACK)

8. Reflection Answers

Q1. Why should networking logic and GUI code be separated?

Keeping networking independent of the GUI improves modularity, maintainability, testing, and code reuse. Changes to the interface do not affect the communication layer.

Q2. Why is a background thread required?

Socket receive operations are blocking. A background thread allows incoming messages to be processed while the main thread remains responsive to user interactions.

Q3. Which components from Assignment 5 were reused?

The TCP server, socket communication, broadcast messaging, private messaging, online user management, chat history, logging, and connection handling were reused with minimal modification.

Q4. How did the GUI improve usability?

The GUI provides a more intuitive interface, making it easier to connect, send messages, monitor online users, and read chat history without memorizing terminal commands.

Q5. Suggest one future enhancement.

A future enhancement would be end-to-end encryption, file sharing, emojis, message persistence using a database, or group chat functionality.

9. Conclusion

The GUI-based multi-client chat application successfully extends the Assignment 5 implementation while preserving the existing networking logic. The application supports multiple clients, broadcast and private messaging, online user management, logging, and responsive communication through multithreading. Testing in Mininet and packet verification using Wireshark confirmed that the application functions correctly and satisfies the assignment requirements.